

STAGE 2 / TAKE-HOME DEVELOPMENT ASSIGNMENT

Build it. Explain it. Show how it works.

A practical implementation exercise based on a design file and assets supplied by Upthrust.

IMPORTANT

There is no advance submission. Complete the assignment before your interview and present it live from your own laptop. At the end of the interview, you will share the project links and a short README.

The task

Build a responsive landing page based on the design file and assets provided by us. You are not expected to redesign the page or create new visual assets. Your job is to translate the supplied design into a polished, production-ready webpage and demonstrate the technical decisions behind your implementation.

Design file[Figma File Link](#)**Assets**[Google Drive Link](#)

Plan for approximately 3–4 hours of focused work and complete the assignment before your scheduled interview.

What we are evaluating

- Accuracy of implementation against the supplied design
- Quality and structure of the frontend build
- Responsive behaviour across desktop, tablet and mobile
- Performance, accessibility and technical SEO
- Forms, conversion tracking and backend/CMS implementation
- Code organisation, version control and deployment quality
- Your ability to explain, debug and modify your own work

WHAT MATTERS

The final visual output matters, but we are equally interested in how the website is built behind the scenes.

IMPLEMENTATION REQUIREMENTS

Build it like a real client project.

1. Match the provided design

Build the page using the supplied design file and assets. Your implementation should closely follow the layout, typography, spacing, colours, components, imagery and responsive behaviour shown in the design.

Where something is not explicitly defined, use reasonable development judgement and be prepared to explain your decision.

2. Responsive implementation

The page should work cleanly across at least:

- 375 px — mobile
- 768 px — tablet
- 1440 px — desktop

The layout should adapt naturally across screen sizes rather than simply scaling down.

3. Performance

Optimise the page for production use. Pay attention to image sizing and compression, lazy loading, font loading, JavaScript payload, third-party scripts and Core Web Vitals.

PERFORMANCE TARGET

Aim for a mobile PageSpeed Performance score of 85 or higher. Be prepared to run or show PageSpeed / Lighthouse results during the interview.

4. SEO & page structure

Implement the basic technical SEO setup expected on a production website:

- Unique page title and meta description
- Exactly one H1 with a logical heading hierarchy
- Canonical URL
- Open Graph title, description and image
- Appropriate structured data where relevant

Important page content should be available in the rendered HTML and should not unnecessarily depend on client-side JavaScript to appear.

5. Accessibility

Build the page with basic accessibility best practices:

- Appropriate image alt text
- Keyboard-accessible interactive elements
- Visible focus states
- Semantic HTML and proper form labels
- Readable colour contrast

ACCESSIBILITY TARGET

Aim for a Lighthouse Accessibility score of 90 or higher.

FORMS, BACKEND & CONTENT

Show us the work behind the page.

Form & conversion tracking

Any form included in the supplied design should be functional. A successful form submission should:

- Validate required fields
- Show a clear success state
- Send or store the submitted information somewhere you can demonstrate
- Trigger a conversion event after a successful submission

REQUIRED EVENT

Push an event named `form_submit` to the Google Tag Manager data layer after a successful submission.

During the interview, you should be able to demonstrate the form working, where the submitted data goes, how the event is triggered, and how you would verify that tracking is working correctly.

Backend / CMS work

We specifically want to see how you structure the website behind the visible webpage. Where appropriate, make key content editable through a CMS, backend or structured content system.

Be prepared to demonstrate how a non-developer could update content such as:

- Headlines and body copy
- Images
- Testimonials
- FAQs
- Repeated content sections

If your solution includes a CMS, database, API, form handler, serverless function or another backend component, configure it properly and be ready to walk us through it.

Be ready to explain

- How content is structured
- How forms are processed
- Where submissions are stored
- How frontend and backend components interact
- How integrations are configured
- How environment variables are handled
- How you would safely update the website after launch

SECURITY NOTE

Do not expose private credentials, passwords, API keys or production secrets during the interview.

STACK, DEPLOYMENT & AI

Use tools you would actually choose.

Choose your own stack

Use the stack you would realistically choose for a client project. There is no preferred framework.

Examples include Webflow, Next.js, Astro, React, Tailwind CSS, a headless CMS, or another comparable production-ready setup.

You should be able to explain:

- Why you chose your stack
- How the project is structured
- What trade-offs you considered
- How your approach would scale beyond this single page

Version control & deployment

The webpage should be available on a live or staging URL so it can be tested during the interview. Vercel, Netlify, Cloudflare Pages and Webflow staging are all acceptable.

If you use code, maintain the project in Git with a sensible commit history. Avoid a single final commit that hides how the work developed.

If you use Webflow, be prepared to show the project structure, class structure, components, CMS collections, forms, custom code and integrations.

Use of AI tools

AI-assisted development tools are allowed and encouraged. You may use tools such as Claude Code, Cursor, GitHub Copilot or ChatGPT.

Be prepared to explain which AI tools you used, what you used them for, and what you reviewed or changed yourself.

AI IS ALLOWED. UNDERSTANDING IS REQUIRED.

Using AI is not a disadvantage. You should understand everything you build because we may ask you to explain or modify parts of the implementation live.

INTERVIEW REVIEW

Present the build live.

You will present the assignment from your own laptop during the interview. There is no advance submission required.

Be prepared to show both the finished webpage and the implementation behind it, including:

- The live webpage
- Desktop, tablet and mobile responsiveness
- Your codebase or Webflow project
- CMS or backend structure
- Form handling and where submissions are stored
- Google Tag Manager / analytics events
- Git history
- SEO implementation
- Accessibility setup
- Performance optimisation
- APIs, integrations or custom code used

Live change

We may ask you to make a small live change during the interview. For example:

- Edit a piece of content or change an image
- Update an FAQ through the CMS
- Adjust a component or fix a responsive issue
- Modify a form event
- Explain or debug a small issue

WHY WE DO THIS

The purpose is not to test how quickly you type. It is to understand how you work and confirm that you can comfortably navigate, explain and modify your own implementation.

At the end of the interview

Please share:

- The live / staging URL
- GitHub repository or Webflow read-only link
- A short README

README should cover

- Stack used and why
- Project structure
- How content can be edited
- CMS / backend setup
- Form handling and tracking
- Any APIs or integrations used
- AI tools used during development
- Known limitations or what you would improve with more time

KEEP IT SHORT

The README does not need to be extensive. It should give another developer or team member enough context to understand, run and continue working with the project.

